



浙江理工大学科技与艺术学院

Keyi College of Zhejiang Sci-Tech University

《单片机课程设计》报告

项目名称： 交通灯控制系统设计

班 级： 24 通信工程专升本 1

姓 名： 傅佳豪

学 号： Zb24681112

浙江理工大学科技与艺术学院

信息与控制学院

目录

一、设计内容和要求.....	3
1.1 设计内容.....	3
1.2 设计要求.....	3
二、设计分析.....	3
2.1 软件设计思路.....	3
2.2 硬件设计思路.....	5
三、系统结构设计.....	5
四、单元电路设计.....	5
五、软件设计.....	6
六、调试.....	20
七、小结.....	24

交通灯控制系统设计

一、设计内容和要求

1.1 设计内容

利用单片机的定时器产生秒信号,控制十字路口的红绿黄灯交替点亮和熄灭,并且用 4 位 LCD 显示器显示十字路口两个方向的剩余时间。要求能用按键设置两个方向的通行时间(绿灯点亮的时间)和暂缓通行时间(黄灯点亮的时间),系统的工作符合一般交通灯控制要求。

1.2 设计要求

- (1) 硬件设计:设计电路原理图,并进行系统功能描述。
- (2) 软件设计:设计程序流程图并编制编程。
- (3) 在软件平台中进行仿真调试。
- (4) 搭建实验电路,下载程序,进行硬件调试。
- (5) 整理实验报告,并对设计过程进行归纳总结。

二、设计分析

2.1 软件设计思路

十字交叉路口的交通灯控制系统的结构如图 1 所示。

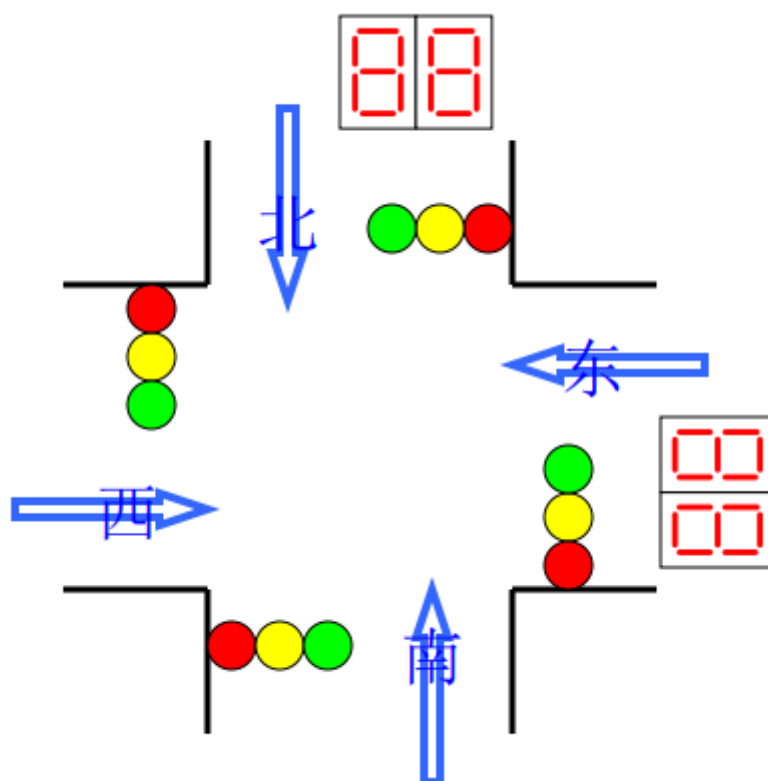


图 1 十字路口交通灯控制示意图

往南和往北的信号一致，即红灯（绿灯或黄灯）同时亮或同时熄灭。用两个 LCD 数码来显示被点亮的指示灯还将点亮多久

往东和往西方向的信号一致，其工作方式与南北方向一样，也采用两个数码来倒计时。当南北方向为绿灯和黄灯时，东西向的红灯点亮禁止通行；而东西方向为绿灯和黄灯时，南北向的红灯点亮禁止通行；当有紧急车辆达到时，路口的信号灯全部变红；当紧急车辆通过后，路口的信号灯变为原来状态。

假设各方向通行时间为 35 秒，绿灯时间为 30 秒，黄灯时间为 5 秒，则其工作方式如表 1 所示循环点亮信号灯。

表 1 交通信号灯工作模式

东西向	绿灯亮 30 秒	黄灯亮 5 秒	红灯亮 35 秒	
南北向	红灯亮 35 秒		绿灯亮 30 秒	黄灯亮 5 秒

2.2 硬件设计思路

单片机采用 MSP430 口袋实验套件，扩展板上包含 8 个 LED 灯和 LCD 显示器，设计时无需其它器件。

- (1) LED 显示系统：可定义南北向和东西向各个三个 LED 灯为绿黄红灯，由 6416 芯片控制。
- (2) 时间倒计时显示系统：南北和东西向各用 2 位 LCD 显示器计时，对该方向的指示灯的点亮时间进行倒计时。
- (3) 键盘系统：4 个按键电路可用通行时间和紧急事件设置。

三、系统结构设计

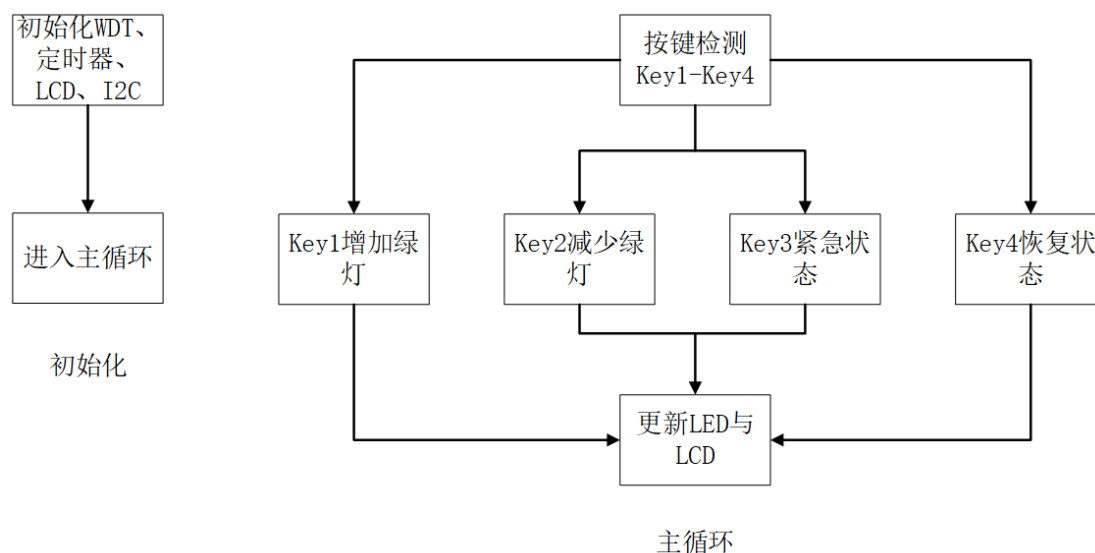


图 2 系统流程图

四、单元电路设计

根据实验平台手册，使用立创 EDA 分别绘制 MSP430G2553 芯片、TCA6416 和 HT1621B 等。原理图如图 2 所示：MSP430G2553 通过 SDA 和 SCL 连接到扩展板上的 TCA6416，通过该 16 位 IO 扩展接口控制扩展板上的 8 个 LED、4 个按键和 LCD 驱动器 HT1621B，HT1621B 连接 LCD_128。

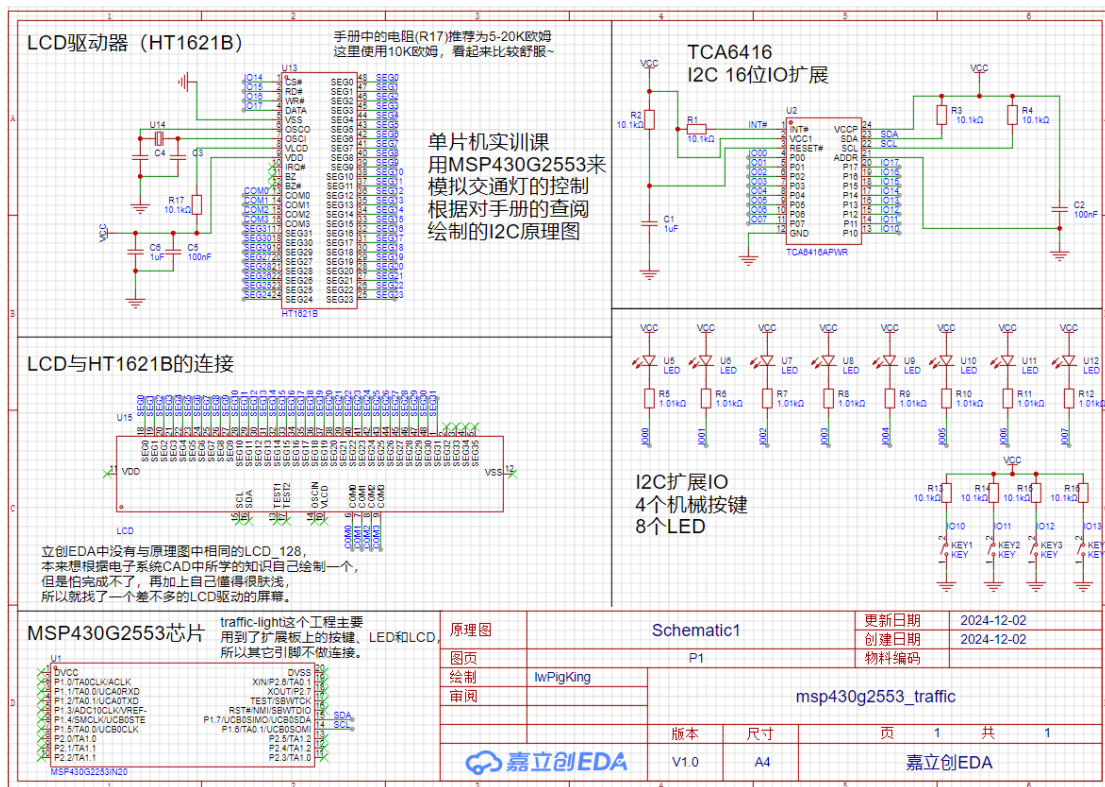


图 3 电路原理图

五、软件设计

/**

* modified by lwPigKing

* description: 浙江理工大学科技与艺术学院单片机实训, 型号为
msp430g2553

* 需求:

- * 1. 二极管模拟东西、南北两个方向的信号灯
- * 2. 东西、南北这两个方向的信号灯工作模式一致, 其中当其中一个方向量。另一个方向就灭
- * 3. 两个方向的信号灯工作周期为 35s
- * 4. 当紧急事件发生的时候, 信号灯全部变成红色
- * 5. 扩展板上的 LED 模拟东西、南北两个方向
- * 6. LCD 数码显示灯还要多久亮
- * 7. Key1 绿灯增 5s, 最大 60s, 超过回到 30s

```
* 8. Key2 绿灯减 5s，最小 10s，小于回到 30s
* 9. Key3 紧急状态键，Key4 解除紧急状态，回到原来的状态
*/
```

```
#include"MSP430G2553.h"
```

```
#include"TCA6416A.h"
```

```
#include"HT1621.h"
```

```
#include"LCD_128.h"
```

```
#include"BCSplus_init.h"
```

```
void I2C_IODect();
```

```
void P13_Onclick();
```

```
void Out_all_redlight();
```

```
void WDT_init();
```

```
void Timer1_ISR();
```

```
void Out_LED(char led_state);
```

```
void Timer_A0_int(void);
```

```
void disp();
```

```
// 显示缓冲
```

```
char dis_buf[4]={0,0,0,0};
```

```
/**
```

```
* 东和西的状态必须一致
```

```
* 南和北的状态必须一致
```

```
* 但是东西、南北的状态不能一致，除非是全红，所以只有如下五中状态
```

S0-4，不符合数学上的组合

```
* S0: 东西绿-南北红
```

```
* S1: 东西黄-南北红
```

```
* S2: 东西红-南北绿
```

```

* S3: 东西红-南北黄
* S4: 紧急状态全红
*
* P1.7,P1.6,P1.5 对应南北的红黄绿灯
* P1.7 对应 LED8, P1.6 对应 LED7, P1.5 对应 LED6
*
* P1.2,P1.1,P1.0 对应东西的红黄绿灯
* P1.2 对应 LED3, P1.1 对应 LED2, P1.0 对应 LED1
* */

// 存放 S0-4 这五个状态的字符数组
char c[5]={0x7e,0x7d,0xdb,0xbb,0x7b};
// 绿灯为 30 秒, 黄灯为 5 秒
char green=30,yellow=5;
// m、n、s 分别为南北与东西转换时间、绿与黄转换时间、状态值
char m=35,n=30,s=0;
// 紧急状态, 0 为非紧急状态
char state_of_emergency = 0;
// 0-不显示
char disp_state= 0;

/**
* LCD、I2C、定时器等, 并启动了低功耗模式 (LPM0) 以及全局中断,
使得系统能够通过中断来响应外部事件 (如按键)。
*/

void main(void) {
    // 关闭哈士奇~~
    WDTCTL = WDTPW + WDTHOLD;
    // 初始化定时器
    BCSplus_graceInit();

```



```
// 初始化 LCD(开启 LCD 驱动)
HT1621_init();

// 初始化 I2C 扩展 IO 单元
TCA6416A_Init();


// 设定 WDT 定时中断为 1000ms，开启 WDT 定时中断使能
// 看门狗定时器
WDT_init();


// 定时器模块初始化
Timer_A0_int();


// 清除屏幕
LCD_Clear();


// 更新显示缓存
HT1621_Reflash(LCD_Buffer);


// 屏幕输出
// 输出 c 数组里的状态
Out_LED(c[s]);


// 低功耗模式
_bis_SR_register(LPM0_bits+GIE);


while(1) {
    PinIN();
    I2C_IODect();
    if(disp_state == 1) disp();
    disp_state = 0;
```

```

        _bis_SR_register(LPM0_bits);
    }
}

/**
 * 设置定时器的的工作模式和启用中断
 */
void Timer_A0_int(void) {
    TACTL = TASSEL_2 + ID_1 + MC_2 + TAIE;
}

/**
 * 忽略了捕获/比较寄存器的中断，并仅关心 Timer_A 溢出中断
 */
#pragma vector=TIMER0_A1_VECTOR
__interrupt void Timer_A(void) {
    switch( TA0IV ) {
        case TA0IV_TACCR1: break; //
CCR1 not used
        case TA0IV_TACCR2: break; //
CCR2 not used
        case TA0IV_TAIFG: _bic_SR_register_on_exit(LPM0_bits); //
overflow,

        break;

        default: break;
    }
}

/**

```

* Key1 - Key4 的功能
* Key1: 增加绿灯持续时间
* Key2: 减少绿灯持续时间
* Key3: 变为紧急状态
* Key4: 从紧急状态变回原来状态
*/

// Key1

void I2C_IO10_Onclick() {

 // 绿灯增加 5 秒

 green = green+5;

 // 超过 60 秒恢复到 30 秒

if(green>60) {

 green=30;

 }

 //根据不同状态，对 m、n 赋值，n-东西绿灯,m-南北绿灯

 // m、n、s 分别为南北与东西转换时间、绿与黄转换时间、状态值

switch(s) {

case 0:

 //m=35, n=30

 m=green+yellow,n=green;

break;

case 1:

 //n=5

 m=yellow,n=yellow;

break;

case 2:

```

        //m=30, n=35

        m=green,n=green+yellow;

        break;

    case 3:

        //m=5

        m=yellow,n=yellow;

        break;

    default: break;

}

}

// Key2

void I2C_IO11_Onclick() {

    // 绿灯减少 5 秒

    green= green-5;

    // 低于 10 秒恢复为 30 秒

    if(green<10) green=30;

    //根据不同状态，对 m、n 赋值，n-东西绿灯或红灯剩余时间,m-南北绿灯

或红灯乘余时间

    switch(s) {

        case 0:m=green+yellow,n=green;break;                //m=35,

n=30

        case 1:m=yellow,n=yellow;break;                    //n=5

        case 2:m=green,n=green+yellow;break;                //m=30, n=35

        case 3:m=yellow,n=yellow;break;                    //m=5

        default: break;

    }

}

```

```
/**
```

* 当 Key3 被按下时，切换至紧急状态。在紧急状态下，所有信号灯会变为红色。

* LED3 和 LED8 亮

```
*/
```

```
void I2C_IO12_Onclick() {
```

```
    state_of_emergency ^= BIT0;
```

```
    if(state_of_emergency) Out_all_redlight();
```

```
}
```

```
/**
```

* Key4 被按下时，继续当前状态

```
*/
```

```
void I2C_IO13_Onclick() {
```

```
    state_of_emergency ^= BIT0;
```

```
    Out_LED(c[s]);
```

```
    dis_buf[0]=n%10;
```

```
    dis_buf[1]=n/10;
```

```
    dis_buf[2]=m%10;
```

```
    dis_buf[3]=m/10;
```

```
    LCD_DisplayDigit(dis_buf[0],2);
```

```
    LCD_DisplayDigit(dis_buf[1],1);
```

```
    LCD_DisplayDigit(dis_buf[2],6);
```

```
    LCD_DisplayDigit(dis_buf[3],5);
```

```
    HT1621_Reflash(LCD_Buffer);
```

```
}
```

```
/**
```

* 该函数通过读取 TCA6416A I/O 扩展器的输入缓冲区，检测按键的按下事件。

* 根据按键的状态变化调用相应的按键处理函数。

* 见 2 实验平台篇（辅助教材）.pdf 的图 1.18 TCA6416A 原理图

* 此函数监测 Key1 (I2C_IO10)、Key2 (I2C_IO11)、Key3 (I2C_IO12)、Key4 (I2C_IO13),

* 分别对应信号灯的时间调节、紧急状态切换和继续状态等功能。

*/

```
void I2C_IODect() {  
    static unsigned char KEY_Now=0;  
    unsigned char KEY_Past;  
    KEY_Past=KEY_Now;  
    //----判断 I2C_IO10 所连的 KEY1 按键是否被按下-----  
    if((TCA6416A_InputBuffer&BIT8) == BIT8)  
        KEY_Now |=BIT0;  
    else  
        KEY_Now &=~BIT0;  
    if(((KEY_Past&BIT0)==BIT0)&&(KEY_Now&BIT0) !=BIT0)  
        I2C_IO10_Onclick();  
    //----判断 I2C_IO11 所连的 KEY2 按键是否被按下-----  
    if((TCA6416A_InputBuffer&BIT9) == BIT9)  
        KEY_Now |=BIT1;  
    else  
        KEY_Now &=~BIT1;  
    if(((KEY_Past&BIT1)==BIT1)&&(KEY_Now&BIT1) !=BIT1)  
        I2C_IO11_Onclick();  
    //----判断 I2C_IO12 所连的 KEY3 按键是否被按下-----  
    if((TCA6416A_InputBuffer&BITA) == BITA)  
        KEY_Now |=BIT2;  
    else
```

```

        KEY_Now &=~BIT2;
    if(((KEY_Past&BIT2)==BIT2)&&(KEY_Now&BIT2) ==0)
        I2C_IO12_Onclick();
    //----判断 I2C_IO13 所连的 KEY4 按键是否被按下-----
    if((TCA6416A_InputBuffer&BITB) ==  BITB)
        KEY_Now |=BIT3;
    else
        KEY_Now &=~BIT3;
    if(((KEY_Past&BIT3) == BIT3)&& (KEY_Now&BIT3) == 0)
        I2C_IO13_Onclick();
}

```

```

/*****
*****
* 名      称: Out_all_redlight()
* 功      能: P1.3 的中断事件处理函数，即当 P1.3 键被按下后，下一步干什么
* 入口参数: 无
* 出口参数: 无
* 说      明: 使用事件处理函数的形式，可以增强代码的移植性和可读性
* 范      例: 无
*****/

```

```

*****/

void Out_all_redlight()
{
    Out_LED(c[4]);
}

```

```

/*****
*****

* 名      称: WDT_init()
* 功      能: 设定 WDT 定时中断为 1000ms, 开启 WDT 定时中断使能
* 入口参数: 无
* 出口参数: 无
* 说      明: WDT 定时中断的时钟源选择 ACLK, 可以用 LPM3 休眠。
* 范      例: 无

*****
*****/

// 看门狗初始化, 1 秒
void WDT_init()
{
    WDTCTL=WDT_ADLY_1000;
    IE1|=WDTIE;
}

/*****
*****

* 名      称: WDT_ISR()
* 功      能: 响应 WDT 定时中断服务,每秒中断一次。
* 入口参数: 无
* 出口参数: 无
* 说      明: WDT 定时中断独占中断向量, 所以无需进一步判断中断
事件, 也无需人工清除标志位。

*          所以, 在 WDT 定时中断服务子函数中, 直接调用 WDT 事件处
理函数就可以了。

```


* 范 例：无

*****/

#pragma vector = WDT_VECTOR

__interrupt void WDT_ISR(void)

{

 disp_state = 1;

}

/******

* 名 称： disp()

* 功 能： 秒计时处理，输出红绿灯状态和显示剩余时间

* 入口参数：无

* 出口参数：无

* 说 明：

* 范 例：无

*****/

void disp()

{

if(!state_of_emergency) //如果是紧急状态，退出中断

{

 m--; //m-1

 n--; //n-1

if(m==0||n==0) //m=0 或者 n=0 时,状态转换

{

 s++;

```

        if(s>3) s=0; //如果状态大于 3，则返回 0

        switch(s) //根据不同状态,对 m、n 赋值, n-东西绿灯或红灯剩余时间,m-南北绿灯或红灯乘余时间
        {
            case 0:m=green+yellow,n=green;break; //m=35, n=30
            case 1:m=yellow,n=yellow;break; //n=5
            case 2:m=green,n=green+yellow;break; //m=30, n=35
            case 3:m=yellow,n=yellow;break; //m=5
            default: break;
        }
        Out_LED(c[s]); //显示 LED 灯
    }
    dis_buf[0]=n%10;
    //dis_buf[0]-dis_buf[3]分别存放 n 的个位十位、m 的个位十位
    dis_buf[1]=n/10;
    dis_buf[2]=m%10;
    dis_buf[3]=m/10;
    LCD_DisplayDigit(dis_buf[0],2);
    LCD_DisplayDigit(dis_buf[1],1);
    LCD_DisplayDigit(dis_buf[2],6);
    LCD_DisplayDigit(dis_buf[3],5);
    HT1621_Reflash(LCD_Buffer); //控制芯片 RAM 更新
}
}

```

```

/**
 * 此函数通过 I2C 总线控制 TCA6416A 扩展 I/O 模块输出相应的信号灯
状态。
 * 通过调用 TCA6416_Tx_Frame() 向 I/O 扩展器发送命令字，从而控制
东西和南北方向的信号灯
 * @Param led_state LED 的状态字节
 */
void Out_LED(char led_state)
{
    unsigned char con[2]={0,0};
    con[0] = Out_CMD0;
    con[1] = led_state;                // 某位置 1，输出为高，0 为低
    TCA6416_Tx_Frame(con,2);          // 写入命令字
}

```

六、调试

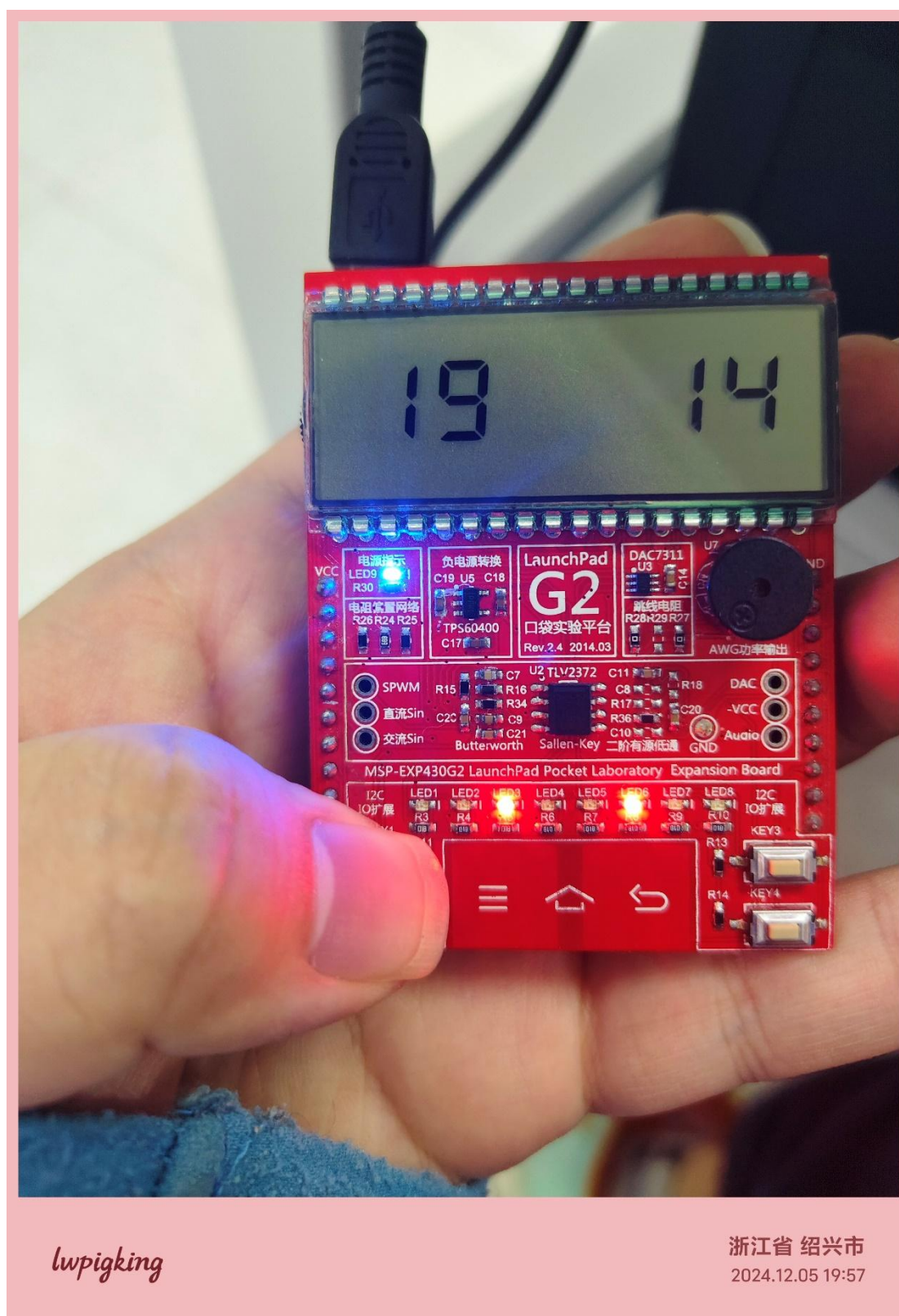
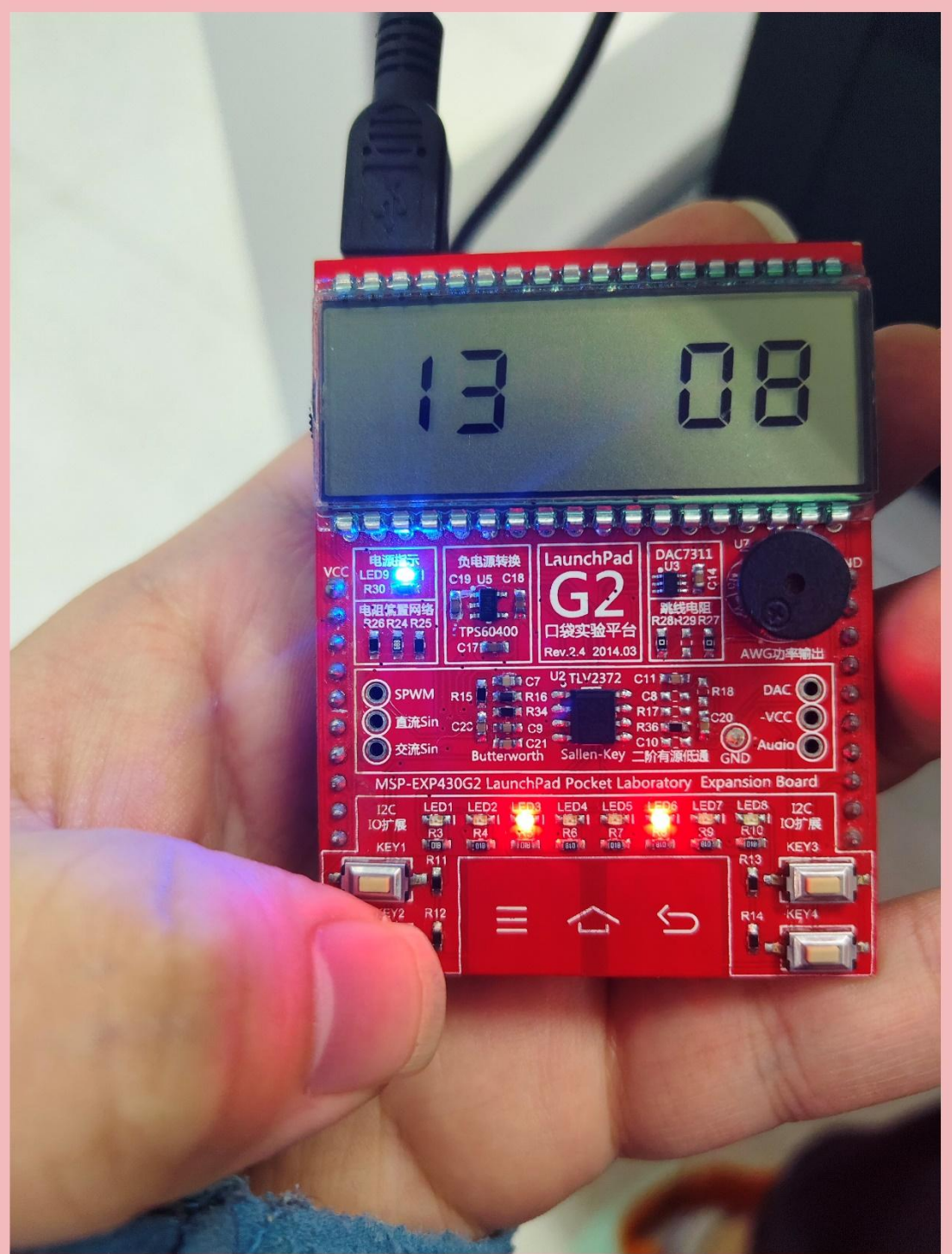


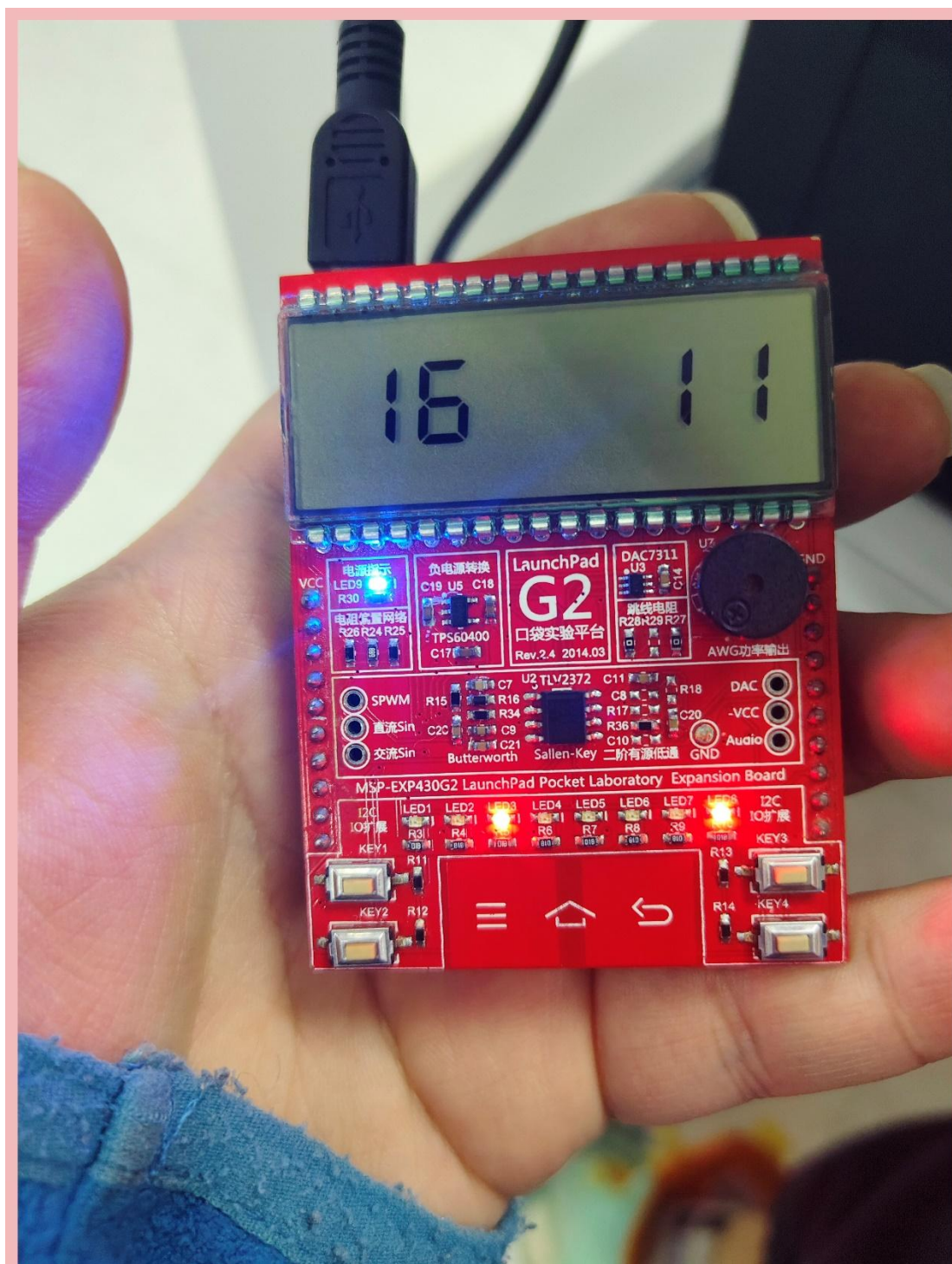
图 4 按下 key1



lw pigking

浙江省 绍兴市
2024.12.05 19:57

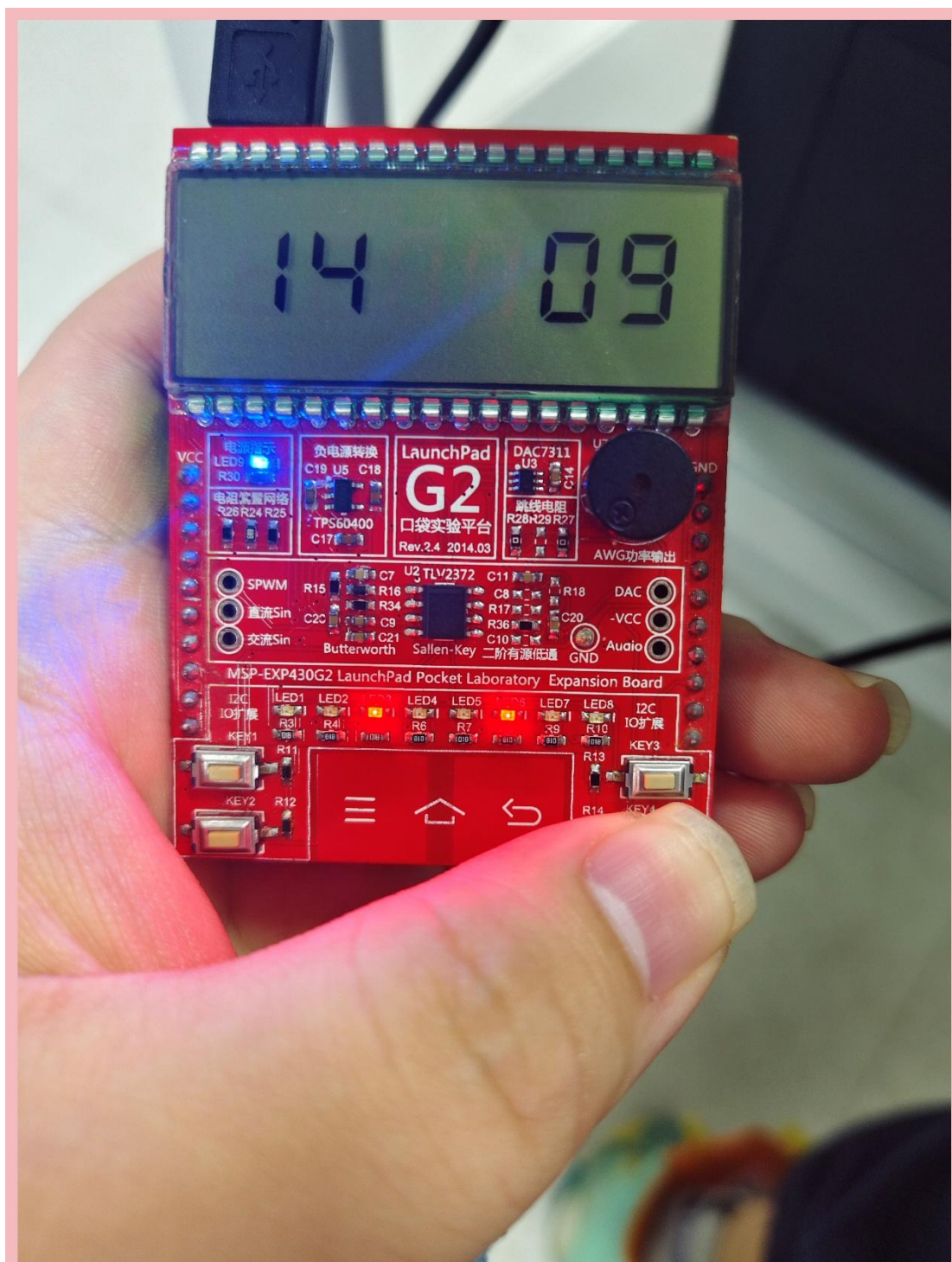
图 5 按下 Key2



lw pigking

浙江省 绍兴市
2024.12.05 19:57

图 6 按下 Key3



lwpigking

浙江省 绍兴市
2024.12.05 19:57

图 7 按下 Key4

七、小结

根据单元电路设计与软件设计,实现了 MSP430G2553 开发板的交通灯工程。
该工程扩展板上的 LED 和 LCD 成功受到按键控制。